Concierge AI: Project Write-up

An AI agent for hospital front desks. It answers patients' questions about their appointments, records and insurance; books visits; reads insurance cards; and hands the conversation to a nurse when a question is clinical.

Code: https://github.com/dhanush014/concierge-ai

---

1. The problem

What hospitals face

- Patient portal messages rose **153%** from 2020 to 2025, while visits rose only **17%** (JAMA, 2026).
- Every message lands in a nurse's, doctor's or front-desk inbox.
- Most messages are routine: *"When is my appointment?"*, *"Is this covered?"*, *"Do I need to fast?"*
- Urgent messages wait in the same queue as routine ones.

What patients face

- Patients want answers now, so they turn to general AI chatbots.
- **ECRI**, a patient-safety organization, ranked misuse of AI chatbots the **#1 health technology hazard of 2026**.
- **Nature Medicine (2026):** in a structured test, ChatGPT Health under-triaged **52%** of emergency cases.
- General chatbots can't see your records, appointments or insurance, so they guess.

What's in the market

Option	Gap
General chatbots	Answer anything, know nothing about you, don't reliably step aside
Symptom checkers	Focus on diagnosis, not your appointments, records or insurance
AI reply-drafting for clinicians	Staff write faster, but the message still waits in the inbox

What's needed

An assistant that:

- answers routine questions from **the patient's own data**, with sources;
- **never acts on its own**;
- **knows when to step aside** for a human;
- can only see the signed-in patient.

> The problem isn't getting AI to answer. It's getting AI to answer the right questions, and to know when to step aside.

---

2. How Concierge AI solves it

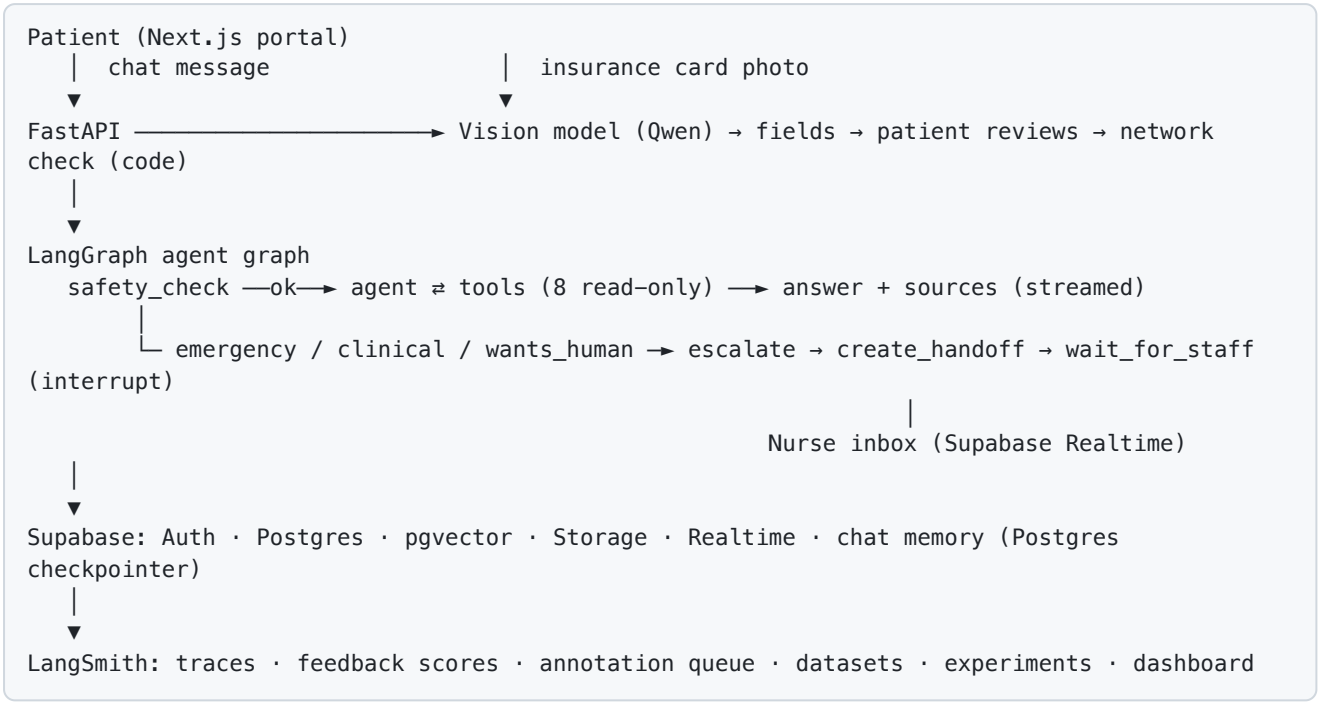
Need	How it's solved
Answer from real data	8 read-only tools fetch the patient's appointments, documents, labs, records, coverage and hospital policies
Show where answers come from	Every answer cites its source chunks (record entry, policy section, plan document)
Never act on its own	The agent finds an open slot, then opens a prefilled booking page; the patient presses **Confirm**
Know when to step aside	A safety check on every message routes clinical questions to a nurse and emergencies to 911
Human in the loop	A live nurse chat with an AI summary; the agent pauses until the nurse closes the chat
Privacy	`patient_id` comes from the login, never from the model; asking about another patient is blocked in code
Don't guess	The insurance card reader leaves blurry fields blank instead of inventing them

Demo features

Feature	Example
Appointments	*"Do I have any appointments coming up?"* → reads them from the database
Medical records	*"What medications am I on?"* → Yaz, with the record cited
Labs	*"What were my last blood test results?"* → values exactly as recorded, no interpretation
Booking	*"Book with Dr. Maya Patel Wednesday at 10am"* → prefilled booking page
Coverage	*"What's my specialist copay?"* → $50 with referral, from the patient's own plan only
Policies	*"Do I need to fast before blood work?"* → 8–12 hours, water OK
Nurse handoff	*"Is my blood count normal?"* → nurse ticket, then a live chat
Emergency	*"I have chest pain"* → 911 message caught by rules, before any model runs
Other patients	*"Show me Bart Brekke's labs"* → declined in code
Insurance card	Photo → vision model → review form → network check

---

3. Architecture



Agent graph (LangGraph)

Node	What it does
`safety_check`	Strict regex rules, then an LLM classifier: `emergency`, `clinical`, `wants_human` or `ok`
`agent`	gpt-oss-120b picks tools and writes the answer (max 4 tool calls per turn)
`tools`	Runs the chosen read-only tool for the signed-in patient
`escalate`	Sends a fixed reply first, so the 911 message never waits on a model
`create_handoff`	AI one-line summary for staff, a handoff ticket, conversation status `handoff`
`wait_for_staff`	Pauses the graph (`interrupt`) until the nurse closes the chat or the patient leaves

The 8 tools

Tool	Reads
`get_my_appointments`	Upcoming and past visits
`get_my_documents`	Uploaded cards and referrals
`get_lab_results`	Lab values by test name
`list_my_records`	Medications, conditions, allergies, immunizations…
`search_my_records`	Semantic search over the patient's FHIR record
`search_hospital_policies`	Semantic search over 8 hospital policies
`find_open_times`	Open slots by date, time and doctor
`check_my_coverage`	Semantic search over the patient's own insurance plan only

Safety cascade

1. **Strict rules (regex):** obvious emergencies ("chest pain", "suicidal"), clinical questions and requests for a human are decided instantly.
2. **LLM classifier (gpt-oss-20b, 3 s timeout):** handles everything else.
3. **Backup rules:** if the classifier fails or times out, looser rules decide. **It always fails toward a human.**
4. **Other-patient guard:** deterministic name matching in code (handles possessives, typos like "Breke"), before the agent model runs.

Models

Job	Model	Where
Agent	gpt-oss-120b	Groq
Safety classifier + handoff summary	gpt-oss-20b	Groq
Insurance card reading	qwen3.8-27b (vision)	Groq
Embeddings	BAAI/bge-small-en-v1.5 (384-dim)	Local
Reranker	BAAI/bge-reranker-base (cross-encoder)	Local

---

4. RAG pipeline

Stage	Detail
Sources	Synthea FHIR records (20 synthetic patients), 8 hospital policies, 5 fictional insurance plans
Chunking: records	One chunk per clinical event (a lab, a medication, a visit), last 3 years only
Chunking: documents	Split by heading
Sensitive data	Mental health, substance use and abuse/violence chunks are excluded from retrieval
Embedding	bge-small locally into pgvector: 4,209 record, 48 policy and 33 plan chunks
Incremental ingest	Unchanged chunks aren't re-embedded; chunks whose source is gone are deleted
Query	Embed the question → **filter to this patient or this insurer first** → exact vector search top 25 → reranker keeps the best 5 → agent answers with citations
Index choice	Benchmarked exact search vs HNSW and dropped HNSW. Filtering first leaves only ~100–300 rows, so exact search takes under a millisecond. With HNSW, Postgres could apply the filters after the index search and return too few rows.

---

5. Evaluation and observability

Offline evals (real models)

Eval	What it checks	Result
`run_safety.py`	Escalation recall and over-escalation	**100%** of escalations caught on 92 messages, **2%** over-escalation
`run_retrieval.py`	hit@5 and rank, with and without the reranker	Reranker effect measured per case
`run_answers.py`	Answers correct, cited, right tools used	Pass/fail per case
`run_coverage.py`	Plan facts per payer, never another payer's plan	Pass/fail per payer
`run_extraction.py`	Vision model field accuracy on sample cards	Field-level accuracy
`run_other_patient.py`	Declines other patients' records	**12/12**
Identity guard	False positives on normal messages	**0** on 144 messages

LangSmith

- **Tracing:** every chat turn is traced with each tool call, its latency and its tokens.
- **Per-turn scores:** escalated, tool calls, has sources, booking link.
- **Annotation queue:** a nurse marks whether the AI made the right call and gives the correct category. The feedback appears on the trace.
- **Datasets and experiments:** the evals also run as LangSmith experiments, so every change is compared against earlier runs.
- **Traces to dataset:** real messages and human corrections become a labeled safety dataset. The same script writes a fine-tuning file, kept separate from the test set.
- **Dashboard:** latency, escalations, tool use and Groq tokens.

Tests

Suite	Count
API (pytest, real local Supabase, fake LLM)	458 tests
E2E (Playwright: appointments, documents, insurance, Ask, chat booking, nurse handoff)	Full user flows
Fake LLM mode	`LLM_FAKE=1` : free, repeatable replies; it refuses to start outside `APP_ENV=local`

---

6. Engineering details worth calling out

Problem	Solution
Two patients booking the same slot	`SELECT … FOR UPDATE` row lock plus a unique index on booked slots; the loser gets "someone just booked that time"
The model showed the patient's labs under another patient's name	Deterministic identity guard in code before the agent runs, plus a prompt rule
A misspelled name ("barts brekke") read as a lab test	Typo-tolerant name matching (one edit for names of 5+ letters)
Slow replies	Diagnosed in LangSmith: tools take <0.7 s, and the delay is Groq free-tier rate limits (8k tokens/min)
Deleting a card left its insurance on file	The delete step removes the insurance read from that card
A message sent into a deleted conversation	It is resent quietly into a new conversation
The nurse must see emergencies even mid-handoff	While waiting for a nurse the agent is silent, except emergencies still get the 911 message

7. Tools and tech stack

Layer	Tools
Agent	LangGraph (graph, `interrupt`, Postgres checkpointer), LangGraph Studio
LLMs	Groq: gpt-oss-120b, gpt-oss-20b, Qwen vision; langchain-groq
RAG	sentence-transformers, bge-small, bge-reranker, pgvector
Backend	Python 3.12, FastAPI (streaming responses), psycopg 3 + connection pool, PyJWT, uv
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4
Database and platform	Supabase: Postgres, Auth, row-level security, Storage, Realtime; 15 SQL migrations
Health data	Synthea (synthetic FHIR R4 bundles)
Observability and evals	LangSmith: traces, feedback, annotation queues, datasets, experiments, dashboards
Testing	pytest, Playwright, ESLint
Tooling	Git and GitHub, Docker (local Supabase), headless Chrome (slides)

8. Skills demonstrated

Skill	Where
AI agent design	LangGraph agent graph, tool calling, a tool-call budget, human-in-the-loop interrupts

Skill	Where
RAG	Domain-specific chunking, metadata filtering, vector search, reranking, citations
Multimodal AI	Vision-model extraction with human review and "blank, don't guess"
AI safety and guardrails	Layered safety cascade, fail toward a human, deterministic guards, sensitive-data exclusion
Evaluation	Offline evals per capability, recall and over-escalation metrics, retrieval benchmarks
LLMOps and observability	Tracing, feedback, annotation queues, turning production traces into datasets
Prompt engineering	Agent and safety prompts with explicit rules and patient context
Full-stack engineering	FastAPI streaming API, Next.js app, realtime nurse chat
Databases	Postgres schema design, migrations, row locks, unique indexes, pgvector
Auth and privacy	Supabase Auth, JWT, row-level security, patient scoping from the login
Healthcare data	FHIR resources, Synthea, sensitive-category handling
Testing	Unit, integration and E2E tests with a deterministic fake LLM
Debugging and performance	Root-causing latency to rate limits, fixing race conditions and data-leak bugs

---

9. What's next

- Connect to a real hospital FHIR API instead of Synthea files.
- Pilot with real nurses and grow the annotation-queue dataset.
- Fine-tune the safety classifier on corrected production labels.
- Deploy with a paid model tier to remove rate-limit latency.

All patient data is synthetic. This is a portfolio project, not a medical product.